

EM algorithm and simulated annealing

Liuxi Mei

Xiaochen Liu

2025-03-03

Question 1: EM algorithm

In the lecture, an EM algorithm was presented for the case of a univariate normal mixture model with two components; you can find it also in the file `emalg.r` on the course homepage.

a. Use the algorithm from the lecture as start and modify it for the case of three components, i.e. the mixture of three normal distributions. Important: Use this provided algorithm to start with and generalize it; do not write completely new code.

```
emalg <- function(dat, eps=0.000001){
  n      <- length(dat)
  pi     <- rep(NA, n)      # initialize vector for prob. to belong to group 1
  pj     <- rep(NA, n)
  # define reasonable starting values for parameters; here based on summary statistics for total dataset
  p1     <- 0.5
  p2     <- 0.25
  p3     <- 1- p1- p2
  # starting value for mixing parameter
  sigma1 <- sd(dat)*1/3      # starting value for standard deviation in group taken as 1/3 of total s
  sigma2 <- sigma1
  sigma3 <- sigma1
  mu1    <- mean(dat)-sigma1/3 # starting values for means, taken a bit lower and a bit higher than over
  mu2    <- mean(dat)+sigma1/3
  mu3    <- mean(dat)
  #initialize parameters
  trace <- list(p1 = c(), p2 = c(), p3 = c(),
               mu1 = c(), mu2 = c(), mu3 = c(),
               sigma1 = c(), sigma2 = c(), sigma3 = c())

  pv     <- c(p1,p2,p3, mu1, mu2, mu3, sigma1, sigma2, sigma3) # parameter vector

  cc     <- eps + 100
  iter <- 0
  # initialize convergence criterion to avoid stopping the while-loop directly
  while (cc>eps){
    pv1  <- pv              # save previous parameter vector
    ### E step ###
    for (j in 1:n){
      pi1 <- p1*dnorm(dat[j], mean=mu1, sd=sigma1)
      pi2 <- p2*dnorm(dat[j], mean=mu2, sd=sigma2)
      pi3 <- p3*dnorm(dat[j], mean=mu3, sd=sigma3)
      pi[j] <- pi1/(pi1+pi2+pi3)
      pj[j] <- pi2/(pi1+pi2+pi3)
```

```

}
### M step ###

p1      <- mean(pi)
p2      <- mean(pj)
mu1     <- sum(pi*dat)/(p1*n)
mu2     <- sum(pj*dat)/(p2*n)
mu3     <- sum((1-pi-pj)*dat)/((1-p1-p2)*n)
sigma1  <- sqrt(sum(pi*(dat-mu1)*(dat-mu1)/(p1*n)))
sigma2  <- sqrt(sum(pj*(dat-mu2)*(dat-mu2)/(p2*n)))
sigma3  <- sqrt(sum((1-pi-pj)*(dat-mu3)*(dat-mu3)/((1-p1-p2)*n)))
#####
# update parameters
trace$p1 <- c(trace$p1, p1)
trace$p2 <- c(trace$p2, p2)
trace$p3 <- c(trace$p3, 1 - p1 - p2)
trace$mu1 <- c(trace$mu1, mu1)
trace$mu2 <- c(trace$mu2, mu2)
trace$mu3 <- c(trace$mu3, mu3)
trace$sigma1 <- c(trace$sigma1, sigma1)
trace$sigma2 <- c(trace$sigma2, sigma2)
trace$sigma3 <- c(trace$sigma3, sigma3)

pv      <- c(p1,p2,1-p1-p2, mu1, mu2, mu3, sigma1, sigma2, sigma3)
cc      <- t(pv-pv1)%*(pv-pv1) # a convergence criterion, maybe not the best one
iter <- iter + 1

}
return(list(parameters = pv, trace = trace))
}
# example data and run of the algorithm
data <- c(0.1, 0.5, 0.7, 1.1, 2.5, 3.4, 3.5, 3.9, 4.0)
emalg(data)$ parameters

```

```
## [1] 0.4441133 0.3674887 0.1883980 0.5996372 3.7336270 2.9220921 0.3604414
## [8] 0.2508682 0.5333941
```

b. Modify the stopping criterion such that the stopping does not depend on a scaling of data. This means that if data in another unit is analyzed, for example if numbers are multiplied or divided by 1000, the results after stopping should be the same.

The convergence criteria is changed as follow:

$$\frac{|p_v^{new} - p_v^{old}|}{|p_v^{old}|} < \epsilon$$

```

emalg2 <- function(dat, eps=0.000001){
  n      <- length(dat)
  pi     <- rep(NA, n)      # initialize vector for prob. to belong to group 1
  pj     <- rep(NA, n)
  # define reasonable starting values for parameters; here based on summary statistics for total dataset
  p1     <- 0.5
  p2     <- 0.25
  p3     <- 1- p1- p2
  # starting value for mixing parameter

```

```

sigma1 <- sd(dat)*1/3          # starting value for standard deviation in group taken as 1/3 of total s
sigma2 <- sigma1
sigma3 <- sigma1
mu1    <- mean(dat)-sigma1/3 # starting values for means, taken a bit lower and a bit higher than ove
mu2    <- mean(dat)+sigma1/3
mu3    <- mean(dat)
#initialize parameters
trace2 <- list(p1 = c(), p2 = c(), p3 = c(),
              mu1 = c(), mu2 = c(), mu3 = c(),
              sigma1 = c(), sigma2 = c(), sigma3 = c())

pv      <- c(p1,p2,p3, mu1, mu2, mu3, sigma1, sigma2, sigma3) # parameter vector

cc      <- eps + 100
iter <- 0
# initialize convergence criterion to avoid stopping the while-loop directly
while (cc>eps){
  pv1 <- pv          # save previous parameter vector
  ### E step ###
  for (j in 1:n){
    pi1 <- p1*dnorm(dat[j], mean=mu1, sd=sigma1)
    pi2 <- p2*dnorm(dat[j], mean=mu2, sd=sigma2)
    pi3 <- p3*dnorm(dat[j], mean=mu3, sd=sigma3)
    pi[j] <- pi1/(pi1+pi2+pi3)
    pj[j] <- pi2/(pi1+pi2+pi3)
  }
  ### M step ###

  p1 <- mean(pi)
  p2 <- mean(pj)
  mu1 <- sum(pi*dat)/(p1*n)
  mu2 <- sum(pj*dat)/(p2*n)
  mu3 <- sum((1-pi-pj)*dat)/((1-p1-p2)*n)
  sigma1 <- sqrt(sum(pi*(dat-mu1)*(dat-mu1)/(p1*n)))
  sigma2 <- sqrt(sum(pj*(dat-mu2)*(dat-mu2)/(p2*n)))
  sigma3 <- sqrt(sum((1-pi-pj)*(dat-mu3)*(dat-mu3)/((1-p1-p2)*n)))
  #####
  # update parameters
  trace2$p1 <- c(trace2$p1, p1)
  trace2$p2 <- c(trace2$p2, p2)
  trace2$p3 <- c(trace2$p3, 1 - p1 - p2)
  trace2$mu1 <- c(trace2$mu1, mu1)
  trace2$mu2 <- c(trace2$mu2, mu2)
  trace2$mu3 <- c(trace2$mu3, mu3)
  trace2$sigma1 <- c(trace2$sigma1, sigma1)
  trace2$sigma2 <- c(trace2$sigma2, sigma2)
  trace2$sigma3 <- c(trace2$sigma3, sigma3)

  pv <- c(p1,p2,1-p1-p2, mu1, mu2, mu3, sigma1, sigma2, sigma3)
  cc <- max((pv-pv1)/(pv1)) # a convergence criterion, maybe not the best one
  iter <- iter + 1
}

```

```

    return(list(parameters = pv, trace2 = trace2))
}

data <- c(0.1, 0.5, 0.7, 1.1, 2.5, 3.4, 3.5, 3.9, 4.0)
emalg2(data)$ parameters

```

```

## [1] 0.4441102 0.3666663 0.1892236 0.5996339 3.7337795 2.9252932 0.3604404
## [8] 0.2508376 0.5347905

```

c. Use the data `dat3p` in `threepops.Rdata` on the course homepage which contains $n = 230$ observations. Create first a histogram of the data. Fit then a normal mixture model with three components using your program from the a. and b. parts. Which estimates do you get for the model parameters?

```

get(load('threepops.Rdata'))

```

```

## [1] 5.705 1.445 6.807 6.655 6.408 2.658 1.495 6.136 7.287 2.526
## [11] 0.546 6.609 6.733 6.488 7.025 6.668 6.517 6.899 7.605 6.657
## [21] 6.455 6.009 6.425 2.177 6.949 7.290 0.592 1.354 2.156 6.339
## [31] 6.858 0.125 6.871 4.233 2.947 7.694 6.192 5.434 6.180 4.298
## [41] 2.954 7.165 -1.227 5.995 6.523 5.946 5.882 3.746 4.228 2.318
## [51] 1.653 4.229 6.773 7.439 3.638 0.829 3.699 4.717 7.952 6.805
## [61] 0.736 1.476 6.539 0.769 6.693 7.538 1.055 7.232 3.232 6.453
## [71] 2.680 6.584 0.882 5.778 5.723 0.614 3.559 3.826 4.072 6.078
## [81] 6.634 3.211 4.690 6.762 3.742 0.415 2.784 7.654 7.746 5.829
## [91] 3.612 7.405 0.101 1.129 0.391 0.557 4.591 7.252 2.825 2.635
## [101] 1.760 7.206 2.347 3.187 4.584 6.069 6.272 5.809 4.246 5.901
## [111] 4.094 4.137 0.855 0.663 7.058 7.090 -2.132 6.355 6.225 6.461
## [121] 4.834 6.457 0.724 6.197 6.953 1.536 3.456 5.769 6.409 0.382
## [131] 4.643 5.493 4.099 5.656 6.717 0.839 3.485 3.334 5.560 6.755
## [141] 6.331 3.583 5.808 6.326 6.019 3.888 7.660 3.816 0.788 6.613
## [151] 4.296 3.353 6.174 5.774 6.013 6.572 7.289 3.699 4.072 6.817
## [161] 0.302 4.011 6.357 0.664 6.284 4.230 0.552 0.238 3.168 7.194
## [171] 6.994 5.401 7.459 7.231 7.293 4.827 5.567 5.930 2.448 6.249
## [181] 1.270 4.964 -0.306 2.933 6.450 7.281 6.028 6.664 6.595 6.504
## [191] 1.339 4.951 6.829 -1.831 6.219 6.112 6.394 4.668 6.847 3.512
## [201] 1.158 4.668 2.464 6.841 2.769 -1.197 4.800 6.141 7.418 6.284
## [211] 5.939 6.828 2.346 8.089 3.250 3.541 3.494 6.667 6.723 6.399
## [221] 6.800 7.482 7.158 7.312 0.679 3.895 3.152 5.696 2.121 4.030

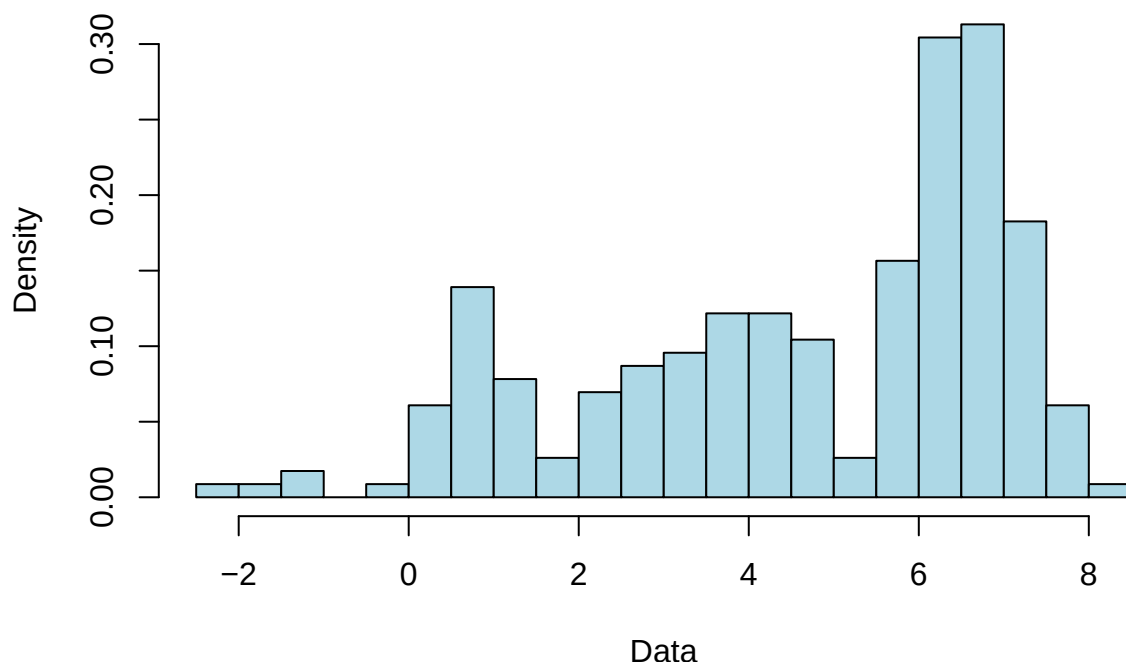
```

```

hist(dat3p, breaks=20, probability=TRUE, main="Histogram of dat3p", xlab="Data", col="lightblue")

```

Histogram of dat3p



```
para1 <- emalg(dat3p)$parameter
para2 <- emalg2(dat3p)$parameter

# Print the estimated parameters
cat("Results from emalg (original stopping criterion):\n")

## Results from emalg (original stopping criterion):
print(para1)

## [1] 0.2262155 0.5247911 0.2489934 1.0676285 6.5817857 3.8303336 1.1950565
## [8] 0.6005079 0.7193544

cat("\nResults from emalg2 (scale-invariant stopping criterion):\n")

##
## Results from emalg2 (scale-invariant stopping criterion):
print(para2)

## [1] 0.2257600 0.5247985 0.2494415 1.0637392 6.5817521 3.8287975 1.1926988
## [8] 0.6005570 0.7202406
```

d. Provide plots of current estimates for each model parameter versus the iteration-number. Do they support that convergence has been achieved for each parameter?

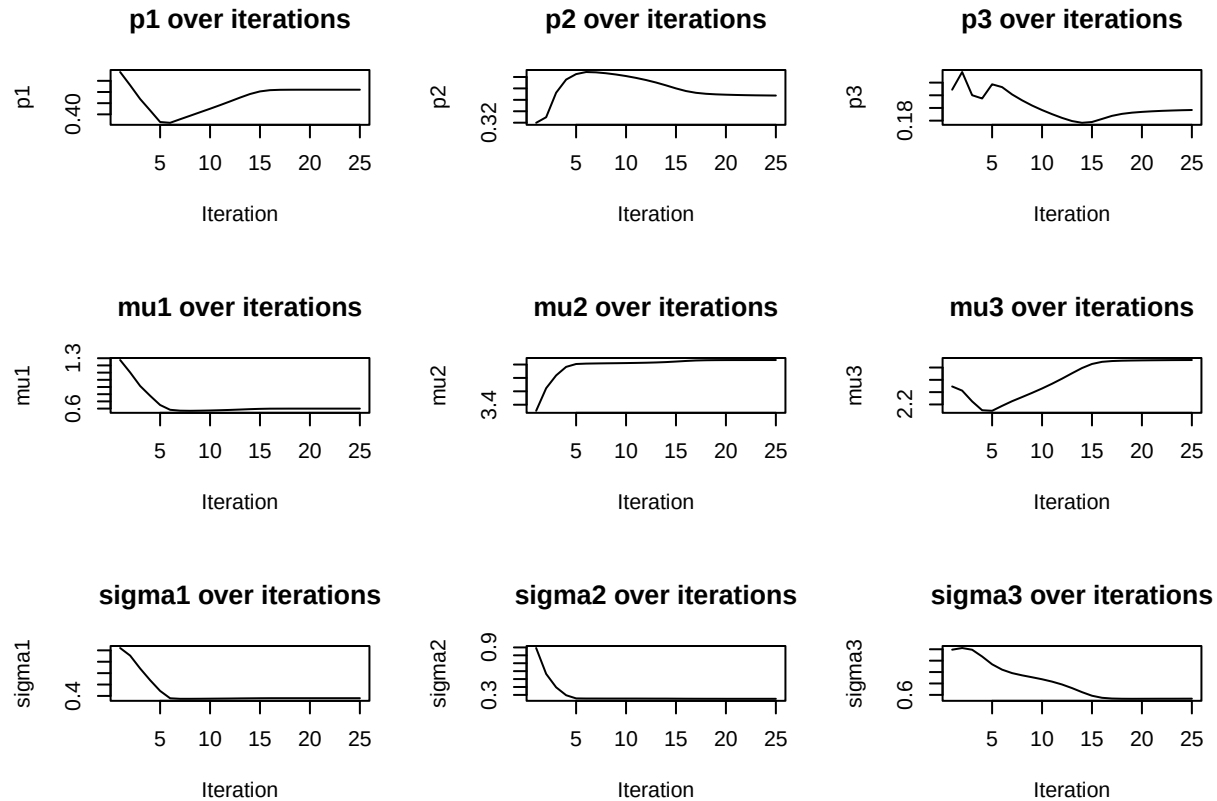
As is shown from the plots, the curves for parameters from both methods seem to stabilize at some level with the increasing of iteration number. Therefore, we think convergences have been reached for each parameters.

```
# Plot convergence
par(mfrow = c(3, 3)) # Create a 3x3 grid for plots
plot(emalg(data)$trace$p1, type="l", main="p1 over iterations", xlab="Iteration", ylab="p1")
plot(emalg(data)$trace$p2, type="l", main="p2 over iterations", xlab="Iteration", ylab="p2")
plot(emalg(data)$trace$p3, type="l", main="p3 over iterations", xlab="Iteration", ylab="p3")
```

```

plot(emalg(data)$trace$mu1, type="l", main="mu1 over iterations", xlab="Iteration", ylab="mu1")
plot(emalg(data)$trace$mu2, type="l", main="mu2 over iterations", xlab="Iteration", ylab="mu2")
plot(emalg(data)$trace$mu3, type="l", main="mu3 over iterations", xlab="Iteration", ylab="mu3")
plot(emalg(data)$trace$sigma1, type="l", main="sigma1 over iterations", xlab="Iteration", ylab="sigma1")
plot(emalg(data)$trace$sigma2, type="l", main="sigma2 over iterations", xlab="Iteration", ylab="sigma2")
plot(emalg(data)$trace$sigma3, type="l", main="sigma3 over iterations", xlab="Iteration", ylab="sigma3")

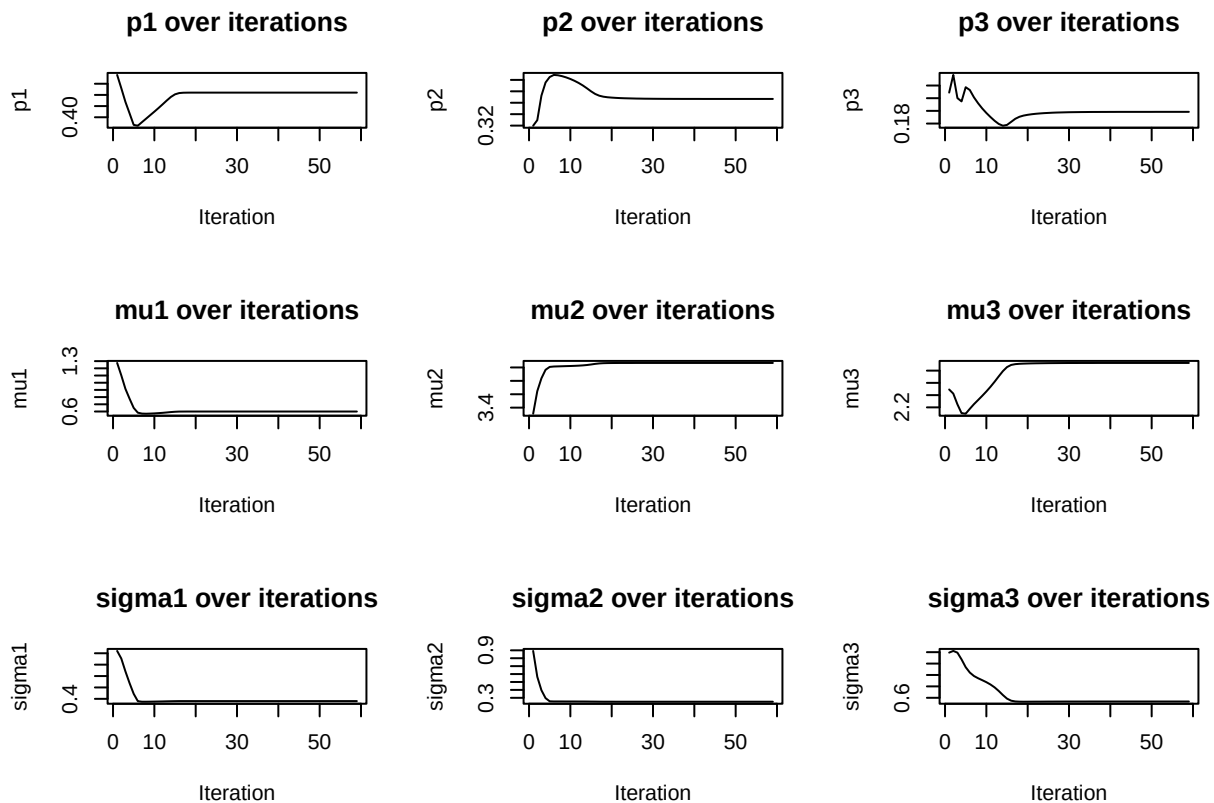
```



```

# Plot convergence
par(mfrow = c(3, 3)) # Create a 3x3 grid for plots
plot(emalg2(data)$trace2$p1, type="l", main="p1 over iterations", xlab="Iteration", ylab="p1")
plot(emalg2(data)$trace2$p2, type="l", main="p2 over iterations", xlab="Iteration", ylab="p2")
plot(emalg2(data)$trace2$p3, type="l", main="p3 over iterations", xlab="Iteration", ylab="p3")
plot(emalg2(data)$trace2$mu1, type="l", main="mu1 over iterations", xlab="Iteration", ylab="mu1")
plot(emalg2(data)$trace2$mu2, type="l", main="mu2 over iterations", xlab="Iteration", ylab="mu2")
plot(emalg2(data)$trace2$mu3, type="l", main="mu3 over iterations", xlab="Iteration", ylab="mu3")
plot(emalg2(data)$trace2$sigma1, type="l", main="sigma1 over iterations", xlab="Iteration", ylab="sigma1")
plot(emalg2(data)$trace2$sigma2, type="l", main="sigma2 over iterations", xlab="Iteration", ylab="sigma2")
plot(emalg2(data)$trace2$sigma3, type="l", main="sigma3 over iterations", xlab="Iteration", ylab="sigma3")

```



Question 2: Simulated annealing

The dataset `bankdata` in `bankdata.Rdata` contains a part of a register of clients who should be contacted to offer a new term deposit. We have here 4364 clients and focus on only two features, the clients' age in years and the logarithm of their account balance (and then multiplied by 6 such that both variables have a similar range, i.e., you would get the original account balance by $\exp(\text{balance}/6)$, but you do not need it for this question, use the two features as provided). The bank wants to investigate how the willingness to agree to the bank's offer depends on the client's age and their account balance. To obtain a first idea about the dependence, 22 clients of these 4364 should be contacted. The bank wants to have a good spread in age and balance in the subsample. Therefore, the bank wants to select the 22 clients in the following way: For each client, we compute the minimal distance to one of the 22 selected clients, where we use the Euclidean distance in the two-dimensional feature space. The sum of these minimal distances for all clients should be minimized. A function `crit` is available in the file `bankcrit.r` on the course homepage which calculates this sum of minimal distances and which you can use.

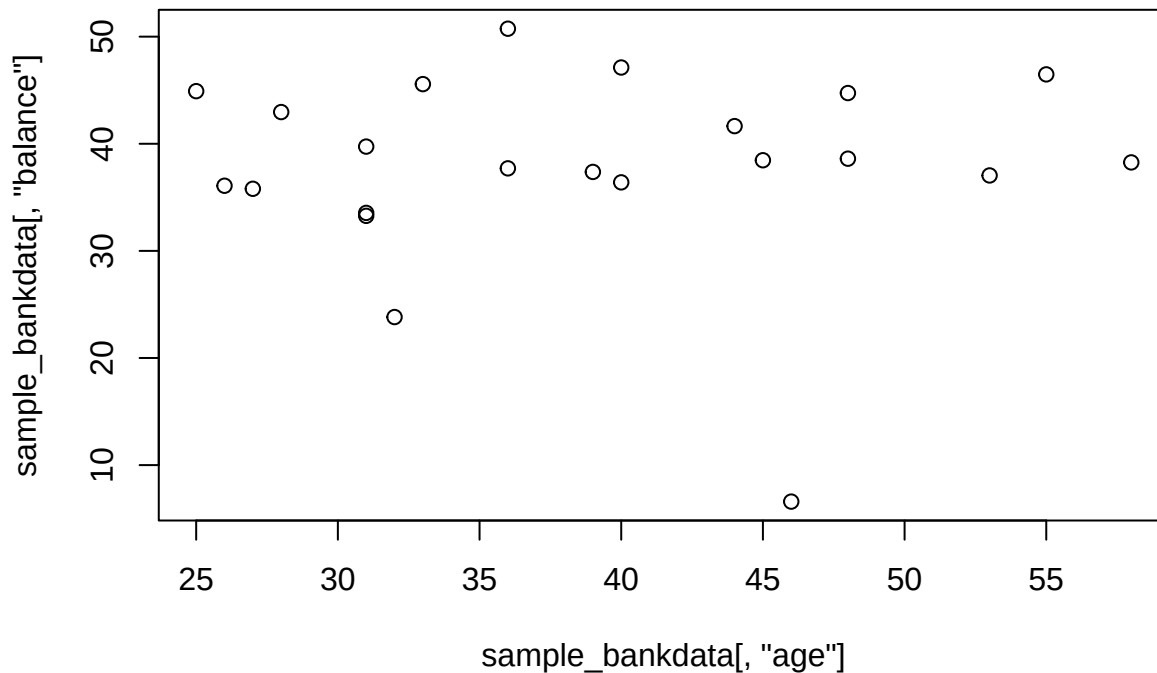
a. Plot the data, select randomly 22 clients and mark them in the plot (this should be your starting subsample for the algorithm in b.).

```
library(tidyverse)

## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.1      v tibble    3.2.1
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.0.4
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
```

```
## x dplyr::lag()      masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
load('bankdata.Rdata')
nclients <- 22

sample_candidate = function(n_samples) {
  sample_index = sample(1:nrow(bankdata),n_samples)
  sample_bankdata = bankdata[sample_index,]
  plot(sample_bankdata[, "age"], sample_bankdata[, "balance"])
  return(list(sample_index=sample_index, sample_bankdata=sample_bankdata))
}
sample_candidate(nclients)
```



```
## $sample_index
## [1] 769 3024 89 861 2584 325 464 2456 2883 768 2124 3298 3533 1781 1943
## [16] 61 4132 2641 3968 3868 3815 1873
##
## $sample_bankdata
##      age  balance
## [1,] 33 45.569306
## [2,] 48 44.735260
## [3,] 36 50.742111
## [4,] 25 44.912950
## [5,] 36 37.704805
## [6,] 31 33.545922
## [7,] 40 47.119298
## [8,] 28 42.960415
## [9,] 39 37.371066
## [10,] 27 35.796880
## [11,] 46 6.591674
## [12,] 58 38.260362
## [13,] 31 33.271065
```



```
## [14,] 40 36.396649
## [15,] 44 41.647140
## [16,] 48 38.607279
## [17,] 31 39.736418
## [18,] 32 23.821751
## [19,] 53 37.042717
## [20,] 55 46.477804
## [21,] 26 36.082291
## [22,] 45 38.461049
```

b. Program your own simulated annealing algorithm to minimize the criterion function. Think about how to select candidates based on the current subset: To ensure that you always have subsets with 22 clients, you might remove and add equally many clients to create the next candidate subset (this means, think about exchanging individuals).

```
crit <- function(dat, subs){
  s <- length(subs)
  nclients <- dim(dat)[1]
  dist <- matrix(rep(NA, nclients*s), ncol=s)
  for (i in 1:s){
    dist[, i] <- sqrt((dat[,1]-dat[subs[i],1])^2+(dat[,2]-dat[subs[i],2])^2)
  }
  return(sum(apply(dist, 1, min)))
}
```

```
get_neighborhood = function(data,current_solution_index) {
  # get all the knn result for all the clients in the bank data
  remove_idx <- sample(1:length(current_solution_index),1)
  removed_client <- current_solution_index[remove_idx]
  new_sample <- current_solution_index[-remove_idx]
  add_candidate <- sample(setdiff(1:nrow(data), new_sample), 1)
  new_sample <- c(new_sample, add_candidate)
  return(new_sample)
}
```

```
exponential_cooling <- function(t0, j) {
  return (t0 * 0.95^j)
}
```

```
linear_cooling <- function(t0, j) {
  return (t0 - 0.1 * j)
}
```

```
simulated_annealing = function(data,n_clients ,beta, initial_temp,func,mj_iterations,max_iterations,earl
  set.seed(123)
  iterations = c()
  crit_values = c()
  init_sample = sample_candidate(n_clients)
  init_solution_index = init_sample$sample_index
  current_solution_index = init_solution_index
  current_target_value = func(data, init_solution_index)
  best_target_value = current_target_value
```

```

previous_best_target_value = current_target_value
best_solution_index = current_solution_index
temperature = initial_temp
m_j = m_j_iterations
stop_count = 0
loop_index = 0
current_temp = temperature
while(current_temp > 0.0001){
  if (loop_index > max_iterations) {
    break
  }
  # cat(paste0("current temp:", current_temp, "\n"))
  # cat(paste0("current m_j:", m_j, "\n"))
  # cat(paste0("current best value:", best_target_value, "\n"))
  # cat(paste0("previous best target value:", previous_best_target_value, "\n"))
  # cat(paste0("current stop count:", stop_count, "\n"))
  # cat(paste0("current loop index:", loop_index, "\n"))
  # cat(paste0("=====", "\n"))
  previous_best_target_value = best_target_value
  for (i in 1:m_j) {
    #new_soultion_index = get_neighborhood(current_soultion_index,knn_res,5)
    new_solution_index = get_neighborhood(data,current_solution_index)
    new_target_value = func(bankdata, new_solution_index)
    delta = new_target_value - current_target_value
    # get a better dist than previous value
    if (delta < 0){
      current_solution_index = new_solution_index
      current_target_value = new_target_value
      if (new_target_value < best_target_value) {
        best_solution_index = new_solution_index
        best_target_value = new_target_value
      }
    }else{
      # get a worse dist than previous value, calcualte the prob to accept the wrose value
      prob = exp(-delta/current_temp)
      if (runif(1) < prob) {
        current_solution_index = new_solution_index
        current_target_value = new_target_value
      }
    }
  }
  loop_index = loop_index + 1
  iterations = append(iterations, loop_index)
  crit_values = append(crit_values, best_target_value)
  stop_count = ifelse(previous_best_target_value == best_target_value, stop_count + 1, 0)

  if (stop_count == early_stop ) {
    break
  }
  # according to the ppt temperature should decrease after each update
  # while m_j should increase
  # temperature = alpha * temperature
  current_temp = cooling_fun(temperature, loop_index)
}

```

```

    m_j = ceiling(beta * m_j)
  }
  return(list(best_solution_index = best_solution_index, best_value = best_target_value, iterations=iterations))
}

```

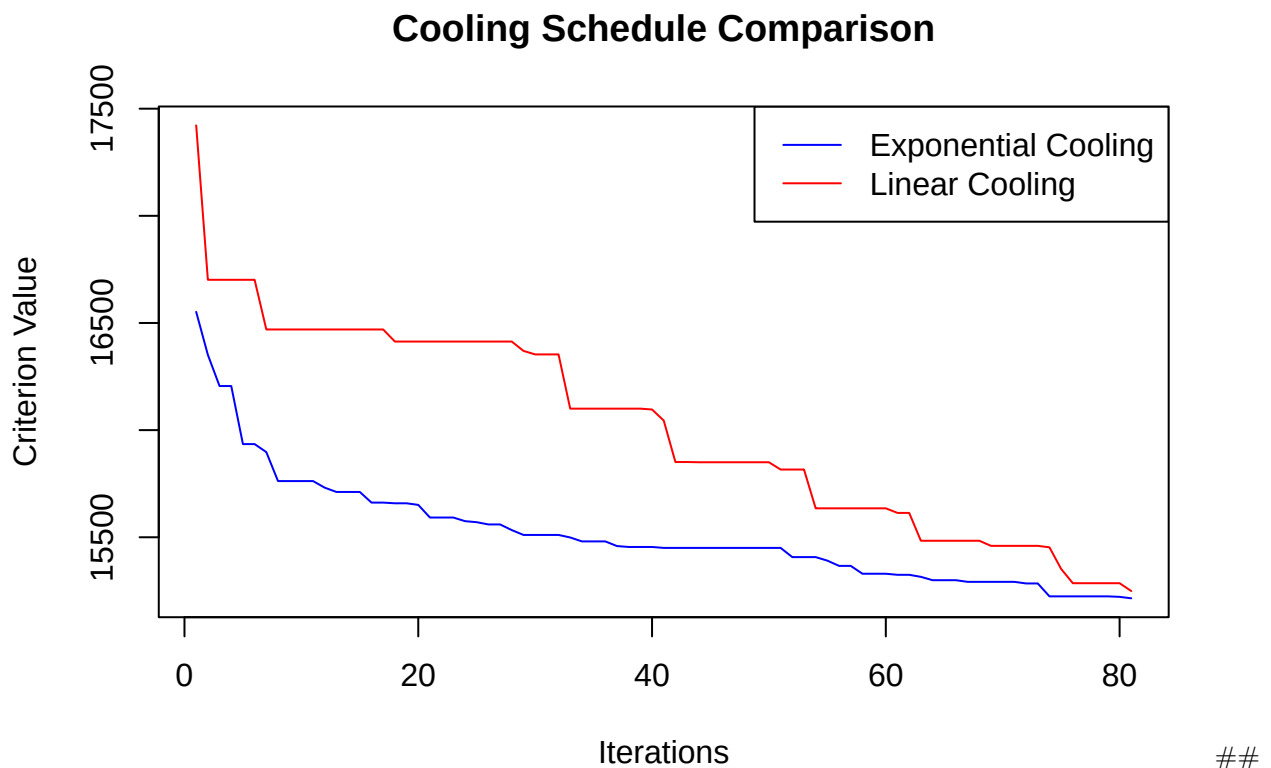
c. Investigate several cooling schedules, starting temperatures, and number of iterations for the simulated annealing algorithm. Discuss at least two combinations of schedules+temperatures+iterations in your report (for example, a good and a bad choice). Compare plots of the criterion-value versus the iteration number. Show also plots of the data with the clients in the final subset marked.

High initial temperature (1000) in `bad_res` results in excessive randomness in the early stage, leading to inefficient exploration. Low initial temperature (10) in `better_res` ensures a more stable starting point, allowing for more efficient optimization. With 1000 iterations per temperature, `bad_res` spends too much time at each temperature level, delaying cooling and reducing efficiency.

```

plot(better_res$iterations, better_res$crit_values, type="l", col="blue",
     xlab="Iterations", ylab="Criterion Value", main="Cooling Schedule Comparison",
     xlim=range(c(better_res$iterations, bad_res$iterations)),
     ylim=range(c(better_res$crit_values, bad_res$crit_values)))
lines(bad_res$iterations, bad_res$crit_values, col="red")
legend("topright", legend=c("Exponential Cooling", "Linear Cooling"), col=c("blue", "red"), lty=1)

```



plot the clients in the final subset

```

final_client = bankdata[better_res$best_solution_index,]
plot(final_client[, "age"], final_client[, "balance"], xlab="Age", ylab="Balance", main="Clients in the f

```

Clients in the final subset

